

Positioning and layers

Most CSS layout you have written so far follows normal flow.

Normal flow is the browser's default way of placing elements: headings stack above paragraphs, block elements create rows, inline elements sit inside text, Flexbox arranges direct children, and Grid creates rows and columns. That should stay your default way to build most layouts.

But some interface pieces do not belong in the normal flow:

- A small label needs to sit on top of an image.
- A header should stay visible while the page scrolls.
- A help link should stay in the corner of the viewport.
- One overlapping piece should appear above another.

Those are positioning problems because a specific element needs to be anchored to a specific reference box.

Start with the visible problem

Use a page with one sticky-looking header, one course card, some scrollable content, and a help link. At first, everything sits in normal flow. That is useful because you can see what positioning changes.

Start with this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Positioning practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <header class="site-header">
      <div class="header-inner">
        <strong>CSS Studio</strong>
        <a href="/lessons">All lessons</a>
      </div>
    </header>

    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Position small pieces without breaking the page</h1>
        <p class="summary">The page layout should stay normal, but a few details need to
      </section>
```



```
<article class="course-card">
  <div class="card-media">
    
    <p class="status-badge">New</p>
  </div>

  <div class="card-body">
    <h2>Build responsive interfaces</h2>
    <p>Learn how layout, spacing, media, and component rules work together in real</p>
  </div>
</article>

<section class="notes">
  <h2>Lesson notes</h2>
  <p>Normal flow is still doing most of the work. The sections stack, the card tak</p>
  <p>Positioning should be saved for the pieces that need special placement, such</p>
  <p>Add enough content here so the page can scroll. That makes the sticky header</p>
  <p>When the page scrolls, notice which elements move with the document and which</p>
</section>
</main>

  <a class="help-link" href="/help">Need help?</a>
</body>
</html>
```

Then add the starting styles:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

a {
  color: #2563eb;
  font-weight: 700;
}

.site-header {
  background-color: white;
  border-bottom: 1px solid #e5e7eb;
}

.header-inner,
.page {
  max-width: 960px;
  margin: 0 auto;
  padding: 0 16px;
}

.header-inner {
  min-height: 64px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}

.page {
  padding-top: 32px;
  padding-bottom: 160px;
}

.intro,
.course-card,
.notes {
  background-color: white;
```

```
border: 1px solid #e5e7eb;
border-radius: 8px;
}

.intro,
.card-body,
.notes {
  padding: 24px;
}

.intro,
.course-card {
  margin-bottom: 24px;
}

.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary,
.course-card p,
.notes p {
  color: #4b5563;
  line-height: 1.6;
}

.card-media img {
  display: block;
  width: 100%;
  height: auto;
}

.status-badge {
  display: inline-block;
  margin: 16px;
  padding: 6px 10px;
  color: #065f46;
  background-color: #d1fae5;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

.help-link {
  display: inline-block;
  margin: 24px;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

Open the page and notice three things:

- The header scrolls away like normal content.
- The **New** badge sits below the image because it is still in normal flow.
- The help link sits after the main content because it is also in normal flow.

That is not broken; it is the browser doing the normal thing.

Positioning starts when you decide that one specific element should leave that normal placement.

Position the badge inside the card image

The badge should belong visually on the image, not below it. **position: absolute** lets you remove that badge from normal flow and place it with offsets such as **top** and **right**.

Update **.status-badge** like this:

CSS

```
.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
}
```

If you open the browser, the badge jumps to the top-right corner of the page instead of the card.



That happens because an absolutely positioned element needs a reference box. If you do not give it a clear positioned ancestor, the browser uses a larger reference area, so **top: 12px; right: 12px** is measured from the page instead of the card image.

To fix that, the image wrapper needs to become the badge's reference box. Give **.card-media** **position: relative**, then the absolute badge can position itself against that parent.

Modify the CSS like this:

CSS

```
.card-media {
  position: relative;
}

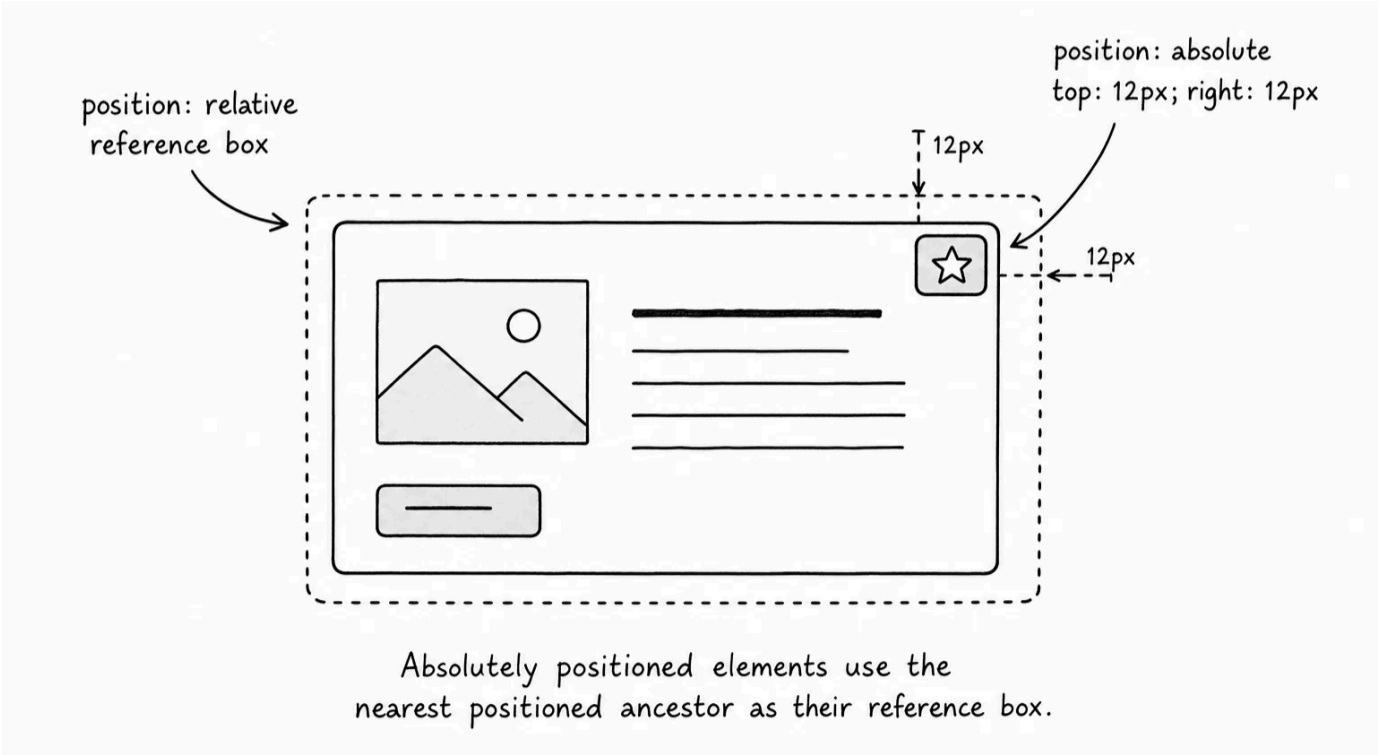
.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
}
```

Now the badge should sit correctly over the image.

`position: relative` does not move `.card-media` by itself. The card is still where it was, but it can now act as the reference box for absolute children.

`position: absolute` removes `.status-badge` from normal flow. The badge no longer takes up a row below the image; it is placed `12px` from the top and `12px` from the right of `.card-media`.

In CSS, a positioned element is any element whose `position` value is not `static`, which is the default.



💡 **Pair absolute with a clear parent** - When you use `position: absolute` on a child, look for the parent that should own the placement. Give that parent `position: relative` so the relationship is obvious.

Keep the header visible while scrolling

The header is part of the page, but it can be useful to keep it visible while the learner scrolls through the content. This is a good job for `position: sticky`.

Make the header sticky:

```
CSS

.site-header {
  position: sticky;
  top: 0;
}
```

`position: sticky` starts like normal flow, so the header still keeps its space at the top of the document.

`top: 0` tells the browser when the sticky behavior should start. When the top of the header reaches the top of the viewport, it sticks there. Without an

offset like `top: 0`, sticky positioning has no threshold to stick to. The browser needs to know where the element should become stuck.

Sticky positioning is useful for headers, table headings, section navigation, and small toolbars. It is not a replacement for normal page layout.

Fix the help link to the viewport

The help link is different from the sticky header.

It should not keep its normal place in the document. It should stay in the bottom-right corner of the browser window even while the page scrolls.

That is a good job for `position: fixed`.

Replace the existing `.help-link` rule with this version:

CSS

```
.help-link {
  position: fixed;
  right: 24px;
  bottom: 24px;
  display: inline-block;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

`position: fixed` removes the help link from normal flow and anchors it to the viewport.

The viewport is the visible browser area. So `right: 24px` and `bottom: 24px` mean the help link sits `24px` from the right and bottom edges of the browser window.

Fixed positioning is common for floating help buttons, cookie banners, persistent mobile actions, and modal backdrops. Use it carefully, because fixed elements can cover content if they are too large or placed in the wrong spot.

Layers decide what appears on top

Positioned elements can overlap. When that happens, the browser needs to know which one should appear closer to the user, and that is what `z-index` controls. Give the positioned elements a small layer scale:

CSS

```
.site-header {
  position: sticky;
```



```
top: 0;
z-index: 10;
}

.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
  z-index: 1;
}

.help-link {
  position: fixed;
  right: 24px;
  bottom: 24px;
  z-index: 20;
  display: inline-block;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

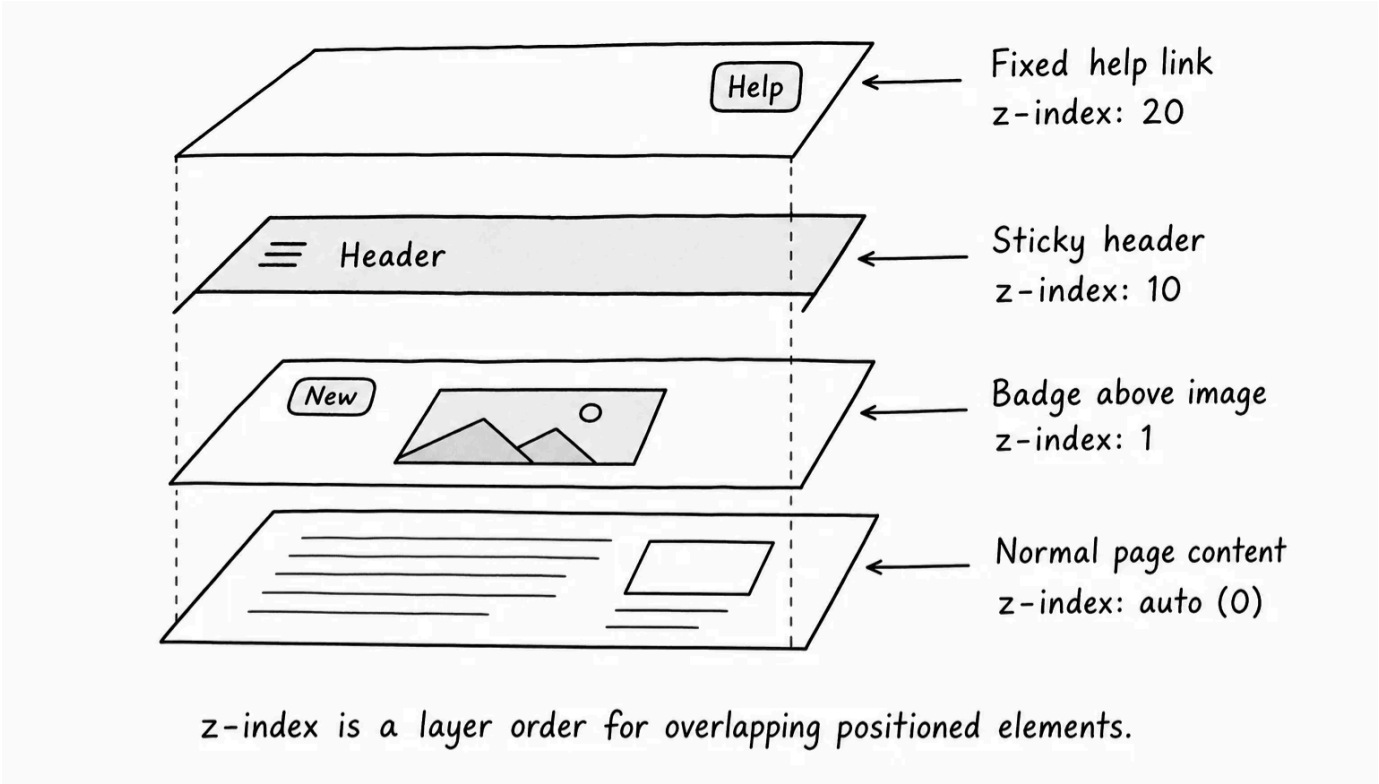
z-index only matters when elements are in layers that can overlap. A higher **z-index** usually appears above a lower one.

Here, the badge only needs to sit above the image, so **z-index: 1** is enough.

The sticky header may overlap page content while scrolling, so it gets **z-index: 10**.

The fixed help link should stay above normal page content, so it gets **z-index: 20**.

The numbers are not magic; they are just a small scale. You usually do not need values like **999999**. If you feel tempted to use a huge number, inspect the elements and find out which layer is actually competing.



Relative offsets are rarely the layout fix

position: relative can also move an element with offsets:

CSS

```
.course-card {  
  position: relative;  
  top: 12px;  
}
```

This moves the card visually, but the space where the card originally belonged is still kept.

The offset works because **.course-card** is positioned. On a normal **static** element, **top**, **right**, **bottom**, and **left** do not move anything.

That can feel strange because the card looks lower, but the surrounding layout still behaves as if the card stayed in its original spot.

For normal spacing between sections, use **margin** or **gap**. Use relative offsets for small visual nudges, not for page layout.

 **Do not use positioning as your main layout system** - If you are arranging a row, a column, a dashboard, or a responsive page, use normal flow, Flexbox, Grid, spacing, and sizing first. Use positioning for anchored details, floating controls, sticky pieces, and deliberate overlaps.

Inspect positioned elements

DevTools makes positioning easier to debug.

Inspect the badge and look at its matched rules. Toggle **position: absolute** off. The badge should return to normal flow below the image.

Turn **position: absolute** back on, then toggle **position: relative** off on **.card-media**. The badge may jump because its reference box changed.

Inspect the sticky header and fixed help link. In the Computed panel, look for **position**, **top**, **right**, **bottom**, and **z-index**.

When positioning does not behave as expected, ask these questions:

- Is the element still in normal flow, or has it been removed from normal flow?
- If it is absolute, which ancestor is its reference box?
- If it is sticky, does it have an offset such as **top: 0**?
- If it is fixed, is it anchored to the viewport?

- If it overlaps another element, does it need a clear `z-index` ?

That debugging habit is better than guessing bigger numbers.

Positioning values in one place

Here is the practical map:

Value	What it is good for
<code>static</code>	The default. The element stays in normal flow.
<code>relative</code>	Keeps the element in normal flow and can create a reference box for absolute
<code>absolute</code>	Removes the element from normal flow and anchors it to a positioned ances
<code>sticky</code>	Starts in normal flow, then sticks to an offset while scrolling.
<code>fixed</code>	Removes the element from normal flow and anchors it to the viewport.

The useful question is not "Which position value should I memorize?"

The useful question is: "What should this element be anchored to?"

Try it

Use the same page and test each positioning idea on purpose:

- Remove `position: relative` from `.card-media` . Notice where the badge goes, then add it back.
- Change the badge from `top: 12px; right: 12px;` to `bottom: 12px; left: 12px;` . Notice how the offsets choose different edges.
- Remove `top: 0` from `.site-header` . Check whether the sticky header still sticks.
- Change `.help-link` from `position: fixed` to `position: absolute` . Scroll the page and compare the behavior.
- Set `.site-header` to `z-index: 0` and `.help-link` to `z-index: 20` . Notice which element stays above if they ever overlap.
- Try moving the card with `position: relative; top: 20px;` , then remove that and use `margin-top: 20px;` . Compare how the surrounding layout behaves.

Positioning is easiest to learn when you keep asking what the element is anchored to. Parent box, viewport, scroll position, or normal flow. Once that is

clear, the CSS choice becomes much less random.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is normal flow?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →